

Edge Sentinel - Hardware Assembly & Coding Guide

v1 reference for bench-side assembly, sensor safety, and code placement

Owner	Mohanees Singh Manral
Repo	github.com/MohaneesH01/edge-sentinel-research-edition
Generated	2026-06-25
Audience	Hardware engineer assembling the v1 node
Tools required	Multimeter, breadboard, jumper wires, USB cable
Bench checklist	docs/Edge_Sentinel_Bench_Checklist.pdf (15 steps)

1. Bill of materials

Item	Qty	Notes
ESP32-WROOM-32 dev board	1	30-pin or 38-pin variant; both work
Adafruit INA219 breakout	1	I ² C default address 0x40
Aosong DHT22 (AM2302)	1	3-pin module with built-in pull-up
Breadboard	1	830 tie-point recommended
Jumper wires (M-M, M-F)	≈20	Mix of lengths; 10cm + 20cm
Solar panel (5-12V)	1	Test panel only; bench supply ok for v1
DC load (resistor or LED array)	1	5W resistor, 10-22 ohm
USB cable (data + power)	1	Micro-USB or USB-C depending on ESP32
Multimeter	1	For voltage / continuity checks

Total v1 BOM cost: ≈\$15-25 single-unit, ≈\$8-12 at 1k volume.

2. The five rules to never break

These rules are in priority order. Breaking any of them can destroy a sensor or the ESP32.

Rule 1: 3.3V only on sensor VCC pins

INA219 and DHT22 both run on 3.3V. Connecting 5V (or, worse, panel voltage) to their VCC pin will kill them. The ESP32's 3.3V output rail is the only safe source.

Rule 2: Panel voltage never reaches the ESP32 GPIO

A 12V panel connected to any ESP32 GPIO will destroy the GPIO. Panel voltage and current must only ever touch the INA219 inputs (VIN+ / VIN- / shunt). The ESP32 talks to the INA219 over I²C only.

Rule 3: INA219 in series with the negative terminal

The INA219 measures current by the voltage drop across its internal 0.1 ohm shunt. Wire it in series with the panel's NEGATIVE terminal going to the load's NEGATIVE. If you put it in series with the POSITIVE terminal, the load's ground reference is no longer at 0V and the ESP32 may see the wrong voltage.

Rule 4: Pull-up on DHT22 DATA pin

DHT22 uses one-wire protocol which requires a 4.7k-10k pull-up resistor from DATA to VCC. The Adafruit DHT22 module has this built-in; bare sensors do not. If you use a bare DHT22, add the pull-up yourself.

Rule 5: Common ground

All grounds must be connected: ESP32 GND, INA219 GND, DHT22 GND, panel negative, load negative. A missing ground connection is the single most common cause of 'weird readings that don't make sense.'

Before plugging in the USB cable, trace the wiring on paper. Confirm Rule 1 (3.3V only) and Rule 3 (INA219 in series with negative). If either is wrong, fix it before powering up.

3. ESP32 pin map for Edge Sentinel v1

Only four GPIOs are used in v1. Everything else is free for v2 upgrades (MPU6050, LoRa, OLED, etc.).

Signal	GPIO	Direction	Notes
I ² C SDA (INA219)	GPIO 21	bidirectional	default ESP32 I ² C SDA
I ² C SCL (INA219)	GPIO 22	output	default ESP32 I ² C SCL
DHT22 DATA	GPIO 4	input	10k pull-up to 3.3V (on-module)
(reserved v2)	GPIO 16, 17	n/a	UART2 for Serial debug overlay

GPIO 21 and 22 are the ESP32's hardware-default I²C pins. Using other pins for I²C requires explicit `Wire.begin(SDA, SCL)` - which the firmware does NOT do in v1. Keep SDA = 21, SCL = 22 unless you're prepared to change both ends.

ESP32-WROOM-32 dev board (30-pin) - relevant pins

3.3V	[V]	[V]	GND	(USB on left, 5V-3.3V-GND rail)
EN	[R]			(reset, leave alone)
GPIO36	[I]	SVP		(input only, NOT used)
GPIO39	[I]	SVN		(input only, NOT used)
GPIO34	[I]			(input only, NOT used)
GPIO35	[I]			(input only, NOT used)
GPIO32	[I]			(touch / ADC, free)
GPIO33	[I]			(touch / ADC, free)
GPIO25	[O]	DAC1		(free)
GPIO26	[I]	DAC2 / ADC2		(free)
GPIO27	[O]	ADC2		(free)
GPIO14	[O]	HSPI CLK		(free)
GPIO12	[O]			(strapping pin, avoid for v1)
GND	[G]			
GPIO13	[O]	HSPI MOSI		(free)
GPIO9	[]	SD2		(flash pin, avoid)
GPIO10	[]	SD3		(flash pin, avoid)
GPIO11	[]	CMD		(flash pin, avoid)
5V	[V]			(USB 5V when USB plugged in)
■■■■		USB / programming port	■■■■	
GND	[G]			
GPIO23	[O]	VSPI MOSI		(free)
GPIO22	[O]	* I ² C SCL *		(INA219 SCL ←)
GPIO1	[O]	TX0		(Serial TX, used for debug)
GPIO3	[I]	RX0		(Serial RX, used for debug)
GPIO21	[O]	* I ² C SDA *		(INA219 SDA →)
GND	[G]			
GPIO19	[O]	VSPI MISO		(free)
GPIO18	[O]	VSPI SCK		(free)
GPIO5	[O]	VSPI CS		(free)
GPIO17	[O]	U2 TXD		(reserved v2)
GPIO16	[O]	U2 RXD		(reserved v2)
GPIO4	[I]	* DHT22 DATA *		(DHT22 DATA ←)
GPIO0	[I]	BOOT		(strapping, leave alone)
GPIO2	[O]			(strapping, avoid)
GPIO15	[O]			(strapping, avoid)
3.3V	[V]	[V]	GND	(right rail)

Green = used by Edge Sentinel v1. Strapping pins (12, 0, 2, 15) and flash pins (6-11) must be left alone unless you know what you're doing.

ASCII schematic (load-side)

Wire-by-wire checklist

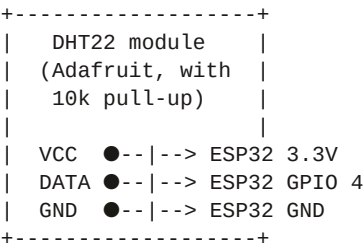
Photo placeholders (replace with real photos after assembly)

[Photo: side view showing panel -> INA219 -> load current path]

5. DHT22 wiring

The DHT22 is the simplest sensor. Three wires: VCC, DATA, GND. The Adafruit DHT22 module has the 10k pull-up built in.

ASCII schematic



Wire-by-wire checklist

From	To	Wire	Rule
DHT22 VCC	ESP32 3.3V	red, short	Rule 1 (3.3V only)
DHT22 DATA	ESP32 GPIO 4	any, short	Rule 4 (pull-up required)
DHT22 GND	ESP32 GND	black	Rule 5 (common ground)

[Photo: DHT22 placement near panel for ambient temperature reading]

6. Power and load

v1 uses a 5-12V solar panel as the source and a 5W resistor (10-22 ohm) as the load. Both connect to the INA219 as described in Section 4.

Why a resistor for the load (not an inverter)?

An MPPT inverter is the right load for a real installation, but it introduces a complex current waveform that's hard to model. For v1 hardware validation we want a known constant load - a 10 ohm 5W resistor gives $I = V/R = 12V/10 = 1.2A$, well within the INA219's $\pm 3.2A$ range. v2 will add an MPPT inverter load.

Load sizing formula

For a panel with $V_{oc} = 22V$ and $I_{sc} = 1.5A$, the safe test load is $R = V_{oc} / I_{sc} * 0.6 \approx 22/1.5 * 0.6 = 8.8 \text{ ohm}$. Use a 10 ohm 5W resistor. For higher-power panels, scale the resistor wattage: $P_{\text{resistor}} \geq V_{oc} * I_{sc} * 0.4$.

Test load options (in priority order)

Option	Resistance	Power	Verdict
10 ohm 5W resistor	10 ohm	5W	✓ for v1
22 ohm 5W resistor	22 ohm	5W	ok for higher-Voc panels
Two 22 ohm in parallel	11 ohm	10W	higher current draw
MPPT inverter	varies	100W+	v2 upgrade
Incandescent bulb	varies	60W	ok but warm

7. Step-by-step assembly (10 minutes)

1. Power down everything

USB unplugged. Panel disconnected. Multimeter on continuity mode.

2. Place the ESP32 on the breadboard

Long sides of the ESP32 plug into the breadboard's power-rail rows. Pin 1 (EN) is on the left when the USB faces away from you.

3. Connect ESP32 3.3V and GND to the breadboard rails

Use two short jumper wires. Red to the red rail (3.3V). Black to the blue rail (GND).

4. Place the INA219 on the breadboard

Leave at least 3 rows of clearance from the ESP32. The breakout has 6 pins: VCC, GND, SCL, SDA, VIN+, VIN-.

5. Wire INA219 power and I²C (Rules 1, 5)

INA219 VCC → 3.3V rail (NOT 5V). INA219 GND → GND rail. INA219 SDA → ESP32 GPIO 21. INA219 SCL → ESP32 GPIO 22.

6. Wire the panel-side current path (Rule 3)

Panel - → INA219 VIN-. INA219 VIN+ → Load -. Load + → Panel + (direct connection, no INA219 involvement).

7. Place the DHT22 on the breadboard

Near the panel for ambient temperature reading. Three pins: VCC, DATA, GND.

8. Wire DHT22 (Rules 1, 4, 5)

DHT22 VCC → 3.3V rail. DHT22 DATA → ESP32 GPIO 4. DHT22 GND → GND rail.

9. Pre-power checks with the multimeter

Continuity from ESP32 GND to every other GND. Continuity from ESP32 3.3V to INA219 VCC and DHT22 VCC. NO continuity from panel + or panel - to ESP32 GND (that would be a short circuit).

10. Plug in USB

If the ESP32 LED comes on, you're powered. Open Arduino IDE and Serial Monitor at 115200 baud. Empty output is normal until you flash firmware.

[Photo: full assembled v1 node on breadboard]

[Photo: 5W resistor load connected to panel]

8. Code map - which line goes where

Each module of Edge Sentinel's code has a single responsibility. When you edit, edit the right file.

Module	File	Lines	Responsibility
Rule-based (Python)	ml/rule_based.py	27	classify_row(): row -> label
Rule-based (C++)	firmware/.../esp32_edge_sentinel.ino	13	classifyFault(): floats -> label
Statistical	ml/statistical.py	≈80	per-class Z-score baseline
Centroid	ml/lightweight_model.py	≈55	argmin Euclidean over centroids
Classical ML	ml/train_classical.py	≈120	DT / RF / IF comparison
Calibration	ml/calibration.py	≈75	temperature scaling
Audit explanations	ml/explain.py	≈80	rule trace + Z attribution + JSON output
MQTT logger	backend/logger.py	≈165	subscribe MQTT → write SQLite
CSV export	backend/export_csv.py	≈35	SQLite -> daily CSV
Dataset generator	backend/generate_dataset.py	≈100	synthetic fault injection
Pipeline	scripts/run_demo_pipeline.py	≈30	end-to-end one-command runner
TinyML export	tinyml/export_centroid_header.py	≈40	centroid JSON -> C header
Dashboard	dashboard/app.py	≈55	Streamlit live monitor
Tests	tests/	≈300	pytest suite, 18 tests

Common edits and where they go

Change a threshold (e.g. open-circuit current)

Two places: ml/rule_based.py and firmware/.../esp32_edge_sentinel.ino::classifyFault(). Keep them in lockstep. The lock is enforced by tests/test_firmware_thresholds.py.

Add a new fault class

1. ml/rule_based.py: add a new condition + label. 2. firmware/.ino: mirror the condition. 3. backend/generate_dataset.py: add a FAULTS class entry + make_reading parameters. 4. docs/research_question.md: update the 6-class taxonomy. 5. docs/firmware_python_alignment.md: update the threshold table.

Change the publish cadence

firmware/.ino: const unsigned long publishIntervalMs = 3000; (milliseconds). v1 default is 3000ms = 3 seconds.

Change the MQTT broker

firmware/esp32_edge_sentinel/config.h: MQTT_HOST, MQTT_PORT, MQTT_TOPIC. config.h is in .gitignore so credentials never reach the repo.

Change the on-device model

Retrain ml/lightweight_model.py: produces tinyml/centroid_model.json. tinyml/export_centroid_header.py: converts JSON to edge_sentinel_centroid_model.h. Firmware includes the header; no firmware code change needed for centroid updates.

Add a new sensor

1. firmware/.ino: add the driver include + initialise in setup() + read in publishTelemetry(). 2. ml/lightweight_model.py: add to FEATURES list. 3. backend/generate_dataset.py: add to make_reading()

parameters. 4. docs/research_question.md: update the fault taxonomy.

9. The 15-step bench checklist (condensed)

Full version: docs/Edge_Sentinel_Bench_Checklist.pdf. This is the condensed version to keep on the bench.

1. Wire ESP32 + INA219 + DHT22 (Sections 4-7 of this PDF).
2. Edit firmware/esp32_edge_sentinel/config.h with WiFi + MQTT.
3. Flash firmware with Arduino IDE (Upload button).
4. Open Serial Monitor at 115200 baud. Confirm 'WiFi connected' and 'MQTT connected' messages.
5. Confirm JSON payload shape: {"v":..., "i":..., "t":..., "p":..., "fault_class":N, "fault_label": "..."}.
6. Capture 1 hour of normal operation. Send the CSV to your software partner.
7. Fault: disconnect load for 10 min (open_circuit).
8. Fault: shade panel for 10 min (partial_shading).
9. Fault: heat DHT22 (temperature_anomaly).
10. Fault: change load to mismatched value (current_deviation).
11. Fault: unplug INA219 SDA wire (sensor_failure).
12. Send all 5 fault CSVs to your software partner.
13. Partner retrains models on real data + regenerates report.
14. Partner sends back the new tinyml header file.
15. Flash new header into firmware; confirm on-device classification.

10. Level upgrade ladder (v1 -> PCB -> STM32)

Edge Sentinel is intentionally staged. v1 is bench-only; v2 ships real-data validation; v3 is a PCB; v4 is an STM32 port. Don't try to skip levels.

Level	Status	What it adds	When
v1 - Bench prototype	Current	ESP32 + INA219 + DHT22 on breadboard, MQTT to InfluxDB, synthetic-data baseline	Nov (Jun-Jul-2026)
v2 - Real-data validation	After bench captures	Replace synthetic with real captures; recalibrate; retrain; regenerate report + TinyML	Aug-Sep-2026
v3 - PCB version	After paper submission	Schematic + layout in KiCad; JLCPCB or PCBWay manufacture; IP65 enclosure; outdoor	Q1-2027
v4 - STM32 port	After v3 works outdoors	Port firmware to STM32F411 (no WiFi - add ESP32 as companion radio); lower power; in	Q2-2027
v5 - Pilot deployment	After v4 validated	5-10 panels at a college lab or small business; Q3-Q4-2027 dashboard; subscription billing set	

What triggers each level upgrade

Trigger	What unblocks
Real-data capture session completed (Step 12 of bench checklist)	v1 -> v2: retrain, regenerate, paper draft ready
IEEE Access submission acknowledged (Aug 2026)	v2 -> v3: invest in PCB; outdoor pilot; KiCad skills
PCB works outdoors for 30 days without failure	v3 -> v4: STM32 port for production-grade reliability
STM32 version validated against ESP32 on same test rig	v4 -> v5: pilot deployment at 5-10 panels
Pilot generates paying customer or LoR from a professor	v5 -> v6: scale to 50+ panels; thesis-grade research

Companion PDFs in docs/: Edge_Sentinel_Bench_Checklist.pdf (15-step detailed plan), project_defense.pdf (interview prep), daily_planner.pdf (Power-BI-style routine).